Secure AI Agent Design Pattern Catalog & Evaluation Framework

Enterprise Architecture Blueprint

---

1. Executive Summary

A platform that catalogs AI agent architectures, documents their trade-offs, scores them against AI-specific security frameworks (OWASP GenAI/LLM Top 10, MITRE ATLAS/ATT&CK), and recommends the right pattern for a given problem via a guided wizard. Three representative patterns (ReAct, Supervisor, RAG Assistant) are fully specified below — diagram, LangGraph implementation, security scoring — as the template every other cataloged pattern follows in the repository (`patterns/<name>/` per §6.4).

---

2. Pattern Catalog

2.1 Catalog Index (all patterns, compact form)

Category	Pattern	Complexity	Typical Security Score*
Single-Agent	Direct Response	Low	90
Single-Agent	Tool-Using Agent	Low-Med	75
Single-Agent	ReAct	Medium	78
Single-Agent	Reflection	Medium	82
Workflow	Sequential Workflow	Low	85
Workflow	Planner-Executor	Medium	70
Workflow	Human-in-the-Loop	Medium	92
Workflow	Approval Workflow	Medium	90
Multi-Agent	Supervisor	High	68
Multi-Agent	Manager-Worker	High	66
Multi-Agent	Collaborative Agents	Very High	58
Multi-Agent	Specialized Agent Teams	Very High	62
Memory	Short-Term Memory	Low	80
Memory	Long-Term Memory	Medium	65
Memory	Knowledge-Augmented Memory	Medium	63
Memory	Episodic Memory	Medium	67

Category	Pattern	Complexity	Typical Security Score*
Enterprise	RAG Assistant	Medium	71
Enterprise	Knowledge Assistant	Medium	74
Enterprise	Documentation Agent	Low-Med	80
Enterprise	Software Engineering Agent	High	60
Enterprise	Business Process Automation Agent	High	64

*Baseline scores assuming no mitigations applied — see §4.3 for scoring methodology. These are catalog defaults; actual score depends on deployment-specific mitigations (§4.4).

2.2 Fully Specified Pattern — ReAct (Single-Agent)

Overview: Interleaves reasoning ("Thought") and tool calls ("Action") in a loop until a final answer is reached. Highest transparency of any single-agent pattern since every step is logged.

```
flowchart LR
    Q[User Query] --> T1[Thought]
    T1 --> A1{Need tool?}
    A1 -- yes --> ACT[Action: call tool]
    ACT --> OBS[Observation]
    OBS --> T1
    A1 -- no --> FIN[Final Answer]
```

LangGraph implementation (representative):

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class ReActState(TypedDict):
    query: str
    scratchpad: list[str]
    final_answer: str | None

def think(state: ReActState) -> ReActState:
    # LLM call: produce next Thought + chosen Action (or final answer)
    ...
    return state

def act(state: ReActState) -> ReActState:
    # Execute the tool named in the last Thought, append Observation
    ...
```

```
    return state

def should_continue(state: ReActState) -> str:
    return "act" if state["final_answer"] is None else END

graph = StateGraph(ReActState)
graph.add_node("think", think)
graph.add_node("act", act)
graph.set_entry_point("think")
graph.add_conditional_edges("think", should_continue, {"act": "act", END: END})
graph.add_edge("act", "think")
app = graph.compile()
```

Advantages: transparent reasoning trace; easy to debug; good tool-selection accuracy. **Disadvantages:** token-expensive (full scratchpad replayed each step); loop can stall on ambiguous tool results. **Suitable use cases:** research assistants, customer support triage, data lookup agents.

Security Considerations & Score

```
Pattern: ReAct
Security Score: 78/100
Risk Assessment:
  - Prompt Injection Risk: Medium   (scratchpad text re-enters the prompt each loop —
                                     an injected Observation can steer subsequent Thoughts)
  - Tool Misuse Risk: High          (agent chooses tools autonomously per step)
  - Data Leakage Risk: Low          (no persistent memory by default)
Recommended Mitigations:
  - Input/Observation sanitization before re-injection into the prompt
  - Tool allowlisting + per-tool permission scopes
  - Context isolation between Observation content and system instructions
  - Human approval layer for any tool call with side effects (write/send/delete)
Mapped Standards: OWASP LLM01 (Prompt Injection), OWASP LLM08 (Excessive Agency),
                  MITRE ATLAS AML.T0051 (LLM Prompt Injection)
```

2.3 Fully Specified Pattern — Supervisor (Multi-Agent)

```
flowchart TB
    U[User Request] --> SUP[Supervisor Agent]
    SUP -->|route| A1[Worker Agent: Research]
    SUP -->|route| A2[Worker Agent: Code]
    SUP -->|route| A3[Worker Agent: Writing]
    A1 --> SUP
    A2 --> SUP
```

```
    A3 --> SUP
    SUP --> OUT[Final Response]
```

Orchestration strategy: supervisor holds shared state, dispatches sub-tasks to workers via typed messages, aggregates results, and decides completion vs. re-delegation. **Failure handling:** per-worker timeout + retry budget; supervisor falls back to a degraded response (partial results + explicit caveat) rather than hanging indefinitely.

Security Considerations & Score

```
Pattern: Supervisor
Security Score: 68/100
Risk Assessment:
  - Agent Impersonation Risk: High   (worker responses trusted by supervisor without
                                      cryptographic origin check by default)
  - Message Tampering Risk: Medium
  - Workflow Manipulation Risk: Medium
Recommended Mitigations:
  - Signed inter-agent messages (HMAC per session key)
  - Supervisor-side schema validation on every worker response before use
  - Least-privilege tool scopes per worker (a "Writing" worker should not hold
    filesystem or network tool access)
Mapped Standards: OWASP LLM Top 10 (2025) — Multi-Agent supplement: Agent Impersonation,
                  Excessive Agency; MITRE ATT&CK T1557 (adjacent: adversary-in-the-middle
                  analog for agent-to-agent channels)
```

2.4 Fully Specified Pattern — RAG Assistant (Enterprise)

```
flowchart LR
    Q[Query] --> EMB[Embed Query]
    EMB --> VDB[(Vector DB - ChromaDB)]
    VDB --> RANK[Rerank + Filter by ACL]
    RANK --> CTX[Build Context]
    CTX --> LLM[LLM Generation]
    LLM --> CITE[Attach Citations]
    CITE --> OUT[Answer]
```

Security Considerations & Score

```
Pattern: RAG Assistant
Security Score: 71/100
Risk Assessment:
```

```
  - Knowledge Base Poisoning Risk: High   (any writable ingestion path is an attack surface)
  - Citation Manipulation Risk: Medium
  - Data Exfiltration Risk: Medium        (cross-tenant retrieval if ACLs aren't enforced
                                             at the vector-store query layer, not just the app
layer)
Recommended Mitigations:
  - Document provenance + integrity hashing at ingestion
  - Row-level ACL filters applied inside the vector query, not post-filtered in app code
  - Citation-to-source verification pass before rendering an answer
Mapped Standards: OWASP LLM03 (Training/Retrieval Data Poisoning), LLM06 (Sensitive
                  Information Disclosure), MITRE ATLAS AML.T0020 (Poison Training Data,
                  analog applied to retrieval corpora)
```

(Every remaining catalog pattern follows this exact template — Overview / Diagram / Implementation / Advantages / Disadvantages / Security Score / Mitigations / Standards mapping — stored one file per pattern under `patterns/<category>/<name>.md` *so the catalog is independently extensible; see §6.4 for the repo layout.)*

---

3. Memory Patterns — Security Summary

Pattern	Storage	Key Risk	Retention Recommendation
Short-Term	In-context / session state	Prompt-injection persistence within a session	Clear on session end; never write to durable store
Long-Term	Vector DB / relational	Memory poisoning, stale-fact reliance	TTL + periodic re-validation against source of truth
Knowledge-Augmented	External KB + retrieval	Retrieval poisoning, citation spoofing	Signed/verified ingestion pipeline only
Episodic	Event log per user/session	Cross-user leakage if keys collide	Strict per-tenant partitioning, encrypted at rest

---

4. Security Evaluation Framework

4.1 Threat Categories Covered

Prompt Injection (direct/indirect/context manipulation/instruction override) · Tool Security (poisoning, malicious outputs, compromised APIs, unauthorized use, excessive permissions) · Memory Security (poisoning, retrieval poisoning, sensitive leakage, cross-user exposure) · RAG Security (malicious documents, KB poisoning, exfiltration, citation manipulation) · Multi-Agent Risks (impersonation, trust, tampering, workflow manipulation).

4.2 Standards Mapping (module design)

Each threat category carries a static mapping table (`security/standards_map.py`) to **OWASP GenAI Top 10**, **OWASP LLM Top 10**, **MITRE ATLAS**, and **MITRE ATT&CK** (for infrastructure-adjacent techniques like credential access via a compromised tool). Mappings are versioned data, reviewed quarterly — not model-generated — for auditability, same principle as the MITRE mapping in the malware-analysis platform design.

4.3 Scoring Methodology

```
security_score = 100
  - Σ (risk_severity_weight × likelihood_weight)  for each unmitigated risk
  + Σ (mitigation_credit)                         for each verified control in place
```

Severity weights: Critical=25, High=15, Medium=8, Low=3. Likelihood weights derived from pattern structure (e.g., autonomous multi-step tool use = higher likelihood of tool misuse than single fixed-tool calls). Mitigation credit only applies to controls the Security Testing Engine can actually verify (§4.4), not ones merely declared in config — this keeps the score honest rather than self-reported.

4.4 Security Testing Engine (backend module)

Runs automated probes against a live agent instance in a sandboxed test harness:

- **Injection probes:** library of adversarial prompts/documents injected at each entry point (user input, tool output, retrieved document); pass/fail on whether the agent's subsequent action changed in an unauthorized way.
- **Tool-permission probes:** attempts out-of-scope tool calls; verifies the agent's tool gateway rejects them server-side.
- **Memory probes:** attempts to plant a poisoned fact and checks whether it influences a later, unrelated session (cross-session leakage check).
- **Multi-agent probes:** injects a spoofed inter-agent message; verifies signature validation rejects it. Each probe run produces a pass/fail + evidence log feeding directly into §4.3's mitigation-credit term.

---

5. Recommendation Engine

Input schema:

```
{
  "problem_statement": "string",
  "data_sensitivity": "public|internal|confidential|regulated",
  "num_users": "int",
  "compliance_requirements": ["GDPR", "HIPAA", "SOC2", "none"],
```

```
  "human_approval_required": "bool",
  "complexity_tolerance": "low|medium|high"
}
```

Decision logic (rule-layer + LLM-justification layer):

1. Hard filters first (deterministic, not model-guessed): `regulated` data sensitivity excludes any pattern without an audit-logged approval workflow; `human_approval_required=true` forces inclusion of a Human-in-the-Loop or Approval Workflow stage regardless of the base pattern chosen.
2. Candidate ranking: score remaining patterns on fit (complexity vs. tolerance, cost estimate vs. scale) plus their §4.3 security score.
3. LLM justification pass: given the top 2-3 candidates and their scores, generate the human-readable "why" — the LLM explains a ranking the rule layer already computed, it does not compute the ranking itself, so recommendations stay auditable rather than a black box.

Output:

```
{
  "recommended_pattern": "Supervisor",
  "alternatives": ["Planner-Executor", "Manager-Worker"],
  "security_risks": ["Agent Impersonation: High", "Message Tampering: Medium"],
  "complexity_estimate": "High",
  "cost_estimate": "$$$ (multi-LLM-call orchestration overhead)",
  "justification": "..."
}
```

---

6. Technical Architecture

6.1 System Diagram

```
flowchart TB
    FE[React + TS + Tailwind + React Flow + Recharts]
    FE --> GW[FastAPI Gateway]
    GW --> AUTH[Auth Service]
    GW --> REPO[Pattern Repository Service]
    GW --> REC[Recommendation Engine]
    GW --> SEC[Security Testing Engine]
    GW --> LGX[LangGraph Execution Service]
    REPO --> PG[(PostgreSQL)]
    REC --> PG
    SEC --> PG
```

```
    LGX --> CHROMA[(ChromaDB)]
    LGX --> LLM[Azure OpenAI]
    SEC --> SANDBOX[Isolated Test Harness]
```

6.2 Database Schema (PostgreSQL, abridged)

```
CREATE TABLE patterns (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL, category TEXT NOT NULL,
    complexity TEXT CHECK (complexity IN ('low','medium','high','very_high')),
    overview TEXT, diagram_mermaid TEXT, langgraph_code TEXT,
    advantages TEXT[], disadvantages TEXT[], use_cases TEXT[]
);

CREATE TABLE security_assessments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    pattern_id UUID REFERENCES patterns(id),
    security_score INT CHECK (security_score BETWEEN 0 AND 100),
    risk_breakdown JSONB,          -- {risk_name: severity}
    mitigations TEXT[],
    standards_mapping JSONB,       -- {owasp_llm: [...], mitre_atlas: [...]}
    assessed_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE enterprise_use_cases (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    pattern_id UUID REFERENCES patterns(id),
    title TEXT, description TEXT, industry TEXT
);

CREATE TABLE evaluation_results (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    pattern_id UUID REFERENCES patterns(id),
    probe_type TEXT,               -- injection | tool_permission | memory | multi_agent
    passed BOOLEAN, evidence JSONB, run_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE recommendations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID, input_payload JSONB,
    recommended_pattern_id UUID REFERENCES patterns(id),
    alternatives JSONB, justification TEXT,
```

```
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE user_feedback (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    recommendation_id UUID REFERENCES recommendations(id),
    rating INT CHECK (rating BETWEEN 1 AND 5), comment TEXT
);
```

6.3 API Design (representative)

Method	Endpoint	Purpose
GET	`/api/v1/patterns`	List/filter catalog (category, complexity, min security score)
GET	`/api/v1/patterns/{id}`	Full pattern detail incl. diagram + code
GET	`/api/v1/patterns/compare?ids=a,b,c`	Side-by-side comparison payload
POST	`/api/v1/security/assess/{pattern_id}`	Trigger Security Testing Engine run
GET	`/api/v1/security/assessments/{pattern_id}`	Latest + historical scores
POST	`/api/v1/recommendation`	Submit wizard input, get recommendation
POST	`/api/v1/langgraph/run/{pattern_id}`	Execute a pattern in the sandboxed Playground
GET	`/api/v1/docs`	Documentation portal content

All write endpoints require analyst/admin role; `/langgraph/run` executes only inside the isolated test harness (§4.4's sandbox), never against production tool credentials.

6.4 Folder Structure

```
agent-pattern-catalog/
├── frontend/
│   ├── src/pages/ (Dashboard, Catalog, ArchitectureViewer, Comparison,
│   │               SecurityCenter, RecommendationWizard, Playground, Docs)
│   ├── src/components/ (PatternCard, MermaidRenderer, ReactFlowDiagram, ScoreBadge)
│   └── src/lib/api.ts
├── backend/
│   ├── app/routers/ (patterns.py, security.py, recommendation.py, langgraph_exec.py, auth.py)
│   ├── app/services/ (recommendation_engine.py, security_testing_engine.py, pattern_repository.py)
│   ├── app/models/ (SQLAlchemy models mirroring §6.2)
│   └── app/security/standards_map.py
├── patterns/
│   ├── single-agent/{direct-response,tool-using,react,reflection}.md
```

```
│   ├── workflow/{sequential,planner-executor,human-in-loop,approval}.md
│   ├── multi-agent/{supervisor,manager-worker,collaborative,specialized-teams}.md
│   ├── memory/{short-term,long-term,knowledge-augmented,episodic}.md
│   └── enterprise/{rag-assistant,knowledge-assistant,doc-agent,swe-agent,bpa-agent}.md
├── langgraph_examples/  # one runnable .py per pattern
└── infra/ (Dockerfiles, k8s manifests, CI pipeline)
```

6.5 UI Pages → Purpose

Dashboard (org-wide score trends) · Pattern Catalog (filterable grid) · Architecture Viewer (React Flow-rendered diagram + LangGraph source) · Pattern Comparison (side-by-side matrix from §2.1) · Security Assessment Center (score breakdown, probe evidence, mitigation checklist) · Recommendation Wizard (the §5 input form) · LangGraph Playground (sandboxed execution, never touching real credentials) · Documentation Portal (this catalog, versioned).

---

7. Threat Model (platform itself, not just the cataloged agents)

```
flowchart LR
    ATT[Attacker] -->|malicious pattern submission| REPO[Pattern Repository]
    ATT -->|adversarial input| PLAY[LangGraph Playground]
    PLAY -->|if unsandboxed| PROD[Production credentials/tools]
    ATT -->|forged recommendation input| REC[Recommendation Engine]
    REPO -->|poisoned langgraph_code field| PLAY
```

Key control: the Playground executes only within an isolated, credential-less sandbox (mirrors the malware-platform's sandbox-isolation principle) — a malicious or buggy pattern submission can never reach a real tool, API key, or production database regardless of what code it contains.

---

8. Deployment & CI/CD

- **Deployment:** Kubernetes; frontend + FastAPI backend as standard deployments; LangGraph Execution Service isolated in its own namespace with no egress except to Azure OpenAI and ChromaDB — never to arbitrary URLs, closing the indirect-prompt-injection-to-SSRF path.
- **CI/CD:** on PR — lint, unit tests, and a mandatory Security Testing Engine run against any modified pattern before merge (a pattern's security score can't regress silently); on merge to main — build, push, staged rollout with the previous version kept warm for instant rollback.
- **Secrets:** Azure Key Vault; no LLM API keys in the Playground's execution context — it uses a scoped, rate-limited proxy key.

---

9. Implementation Roadmap

Phase	Scope
1	Pattern Repository + static catalog (read-only), 3 fully-specified patterns live
2	Security Testing Engine (injection + tool-permission probes) + scoring
3	Recommendation Engine (rule layer + LLM justification)
4	LangGraph Playground (sandboxed execution)
5	Full catalog completion (remaining 17 patterns to the §2.2 template) + memory/multi-agent probes
6	Enterprise rollout: SSO integration, audit export, compliance reporting views

10. Future Enhancements

Continuous red-teaming pipeline that auto-generates new injection probes from disclosed real-world LLM CVEs/incidents; per-organization custom pattern submissions with a review gate before catalog inclusion; drift detection that re-scores a pattern's security rating when its underlying model or tool dependency changes.